

Fundamentals of Data Science

Lectures 6 - Statistical Modelling Part I

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2025

¹Co-founder Quantum Leap, ex-QuantCo

²Data Scientist/ML Engineer @ QuantCo

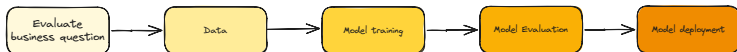
Table of contents

1. The pillars of effective modelling
2. Choice of modelling approach
3. Going beyond the simple linear model
4. Scikit-learn's estimator API
5. Cross-Validation
6. Case study: French Motor Third-Party Liability Claims
7. Weekly dose of SWE: Testing & Debugging

The pillars of effective modelling

The pillars of effective modelling - data quality is key

The model can only be as good as its ingredients:



Note

If we build a model based on a wrong understanding of the business objective or on low-quality data, any time spent on trying improve model metrics using feature engineering or tuning is likely worthless.

Just imagine trying to model conversion rates, while we missed 10% conversions due to a mistake in some prior lookup in our data pipeline.

The pillars of effective modelling - know your data

Before simply throwing some estimator on our data, we should have:

- Examined the target variable properly (distribution, outliers, etc.).
- Checked for any systematically missing observations.
- A good understanding of the data generating process.
- Have thought about potential data drift.

Examples: Insurance claims modelling

- We might systematically miss very large claims in our training data, which take long to settle.
- The target variable might contain actual claims as well as reservings.

The pillars of effective modelling - know your constraints

Things we should try to clarify early on:

- Which features can we use?
 - Is this feature available at the time and point of inference?
 - Are we allowed to use this feature or does it contain sensitive or discriminating information?
- Do we have to incorporate constraints?
 - Do we need to incorporate monotonicity constraints?
 - Do we face interaction constraints of features (e.g. linearity in prices rel. to some feature)

The pillars of effective modelling - choose your objective function

We typically project our target variable Y onto some feature space to get an approximation of $f(X)$. Well, what is a good projection for the underlying task we face?

- What distribution mimics the target variable best? (e.g. normal, log-normal, poisson, gamma, etc.)
- Do we care about the mean (e.g. MSE objective) or about specific quantiles (e.g. MAE objective for median, or asymmetric loss for other quantiles)?
- Do we want the projection to be robust against outliers (e.g. Huber loss)?
- Do we care more about percentage errors or errors at the scale of the target?
- ...

The pillars of effective modelling - choose your objective function

Policy functions informed by estimators

In industry settings we are also often interested in policy functions which are informed by estimators. So, given some estimate we might want to optimize some business KPI.

Example: Take the example of pricing products for some e-commerce business. In a first step we might be interested in a good estimate of customer demand to get a better idea of how price elastic the market or specific customer groups are. To arrive at a pricing function, we could then optimize profits using our demand estimation and cost information.

Choice of modelling approach

Finding the right approach for the task at hand

After having spent a lot of time on data retrieval, cleaning and pre-processing, it might be tempting to just plug the data into some model. We should, however, make an informed decision about the modelling approach we choose.

To guide this decision, we will shed some light on:

- Parametric vs. Non-Parametric Methods
- Supervised vs. Unsupervised Learning
- Regression vs. Classification
- Prediction vs. Causal Identification
- Trade-off between prediction accuracy and interpretability

Parametric vs. Non-Parametric

Parametric methods typically involve a two-step model approach:

1. We have to make an assumption about the functional form a priori. One such assumption could be that the model is linear in X :

$$f(x) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

While the model is linear in X , X can contain non-linear transformations of features.

2. Now the problem reduces to **estimating the parameters**, e.g. via OLS or Maximum Likelihood.

Advantage: easy to estimate, more efficient;

Disadvantage: manual parametrization, usually won't match unknown form of f ;

We can try to make the regressors in X as flexible as possible, but more **complex models can also quickly lead to overfitting**.

Parametric vs. Non-Parametric

Non-parametric models try to fit the data points as closely as possible without making an assumption about the functional form.

Because they fit f directly instead of reducing it to a vector of parameters, they need a lot of data points to get an accurate estimate of f .

Different notions of non-parametric

Some non-parametric methods, such as tree-based methods, dynamically construct splits which can be interpreted as finding a parametrization in the form of many piece-wise constant functions.

Other methods, such as kernel methods, or e.g. nearest neighbour rely directly on the data. We will focus here on the former when we speak about non-parametric methods.

Supervised vs. Unsupervised

Most of the time we are interested in **predicting or understanding some outcome variable y given some covariates X** , hence we are in a supervised setting, where we project y onto the feature space.

Unsupervised learning faces the challenge of trying to **find relationships in the data without any labels**, e.g. cluster analysis. Bot detection is often a use case where we often don't have labels, but still try to find clusters which show some anomaly.

Regression vs. Classification

The dichotomy between regression and classification is not as stark as sometimes presented, as **classification simply focusses on the domain $[0,1]$ to predict qualitative labels** (e.g. conversion).

Recall the logistic regression or its multinomial version, which aims at predicting probabilities to belong to a specific group.

Prediction vs. Causal Identification

Prediction aims to understand the relationship of the **outcome variable for different combinations of the input variables**: $\hat{Y} = \hat{f}(X)$.

Causal inference is interested in the **change of outcome given some treatment within the same unit** of observation: $\frac{\Delta \hat{y}_i}{\Delta x_i} = \hat{\beta}(x_i)$.

Conversion example

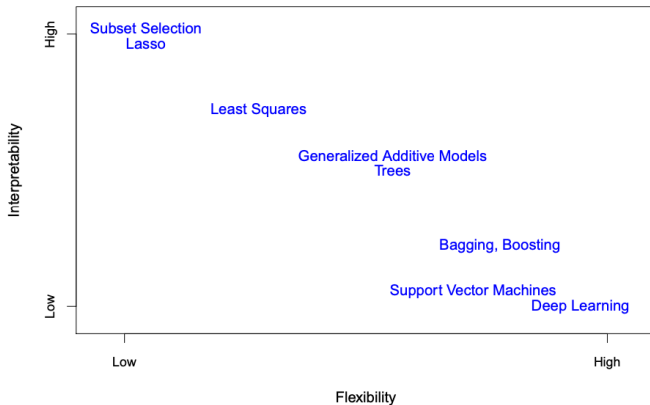
We can have a model with very good predictive accuracy of whether customers will buy, given their characteristics and the price they have to pay and potentially more features such as time of year/week/day.

However, this does not imply that we have an accurate estimate of the causal effect of the price on conversion as we face endogeneity. We need some exogenous price signal to measure how individuals of similar characteristics, at the same point in time, react to the price.

Example of elasticity estimation: [Link](#)

Trade-off between prediction accuracy and interpretability

There is a clear trade-off between prediction accuracy, which typically comes at the cost of model complexity, and interpretability:



Source: [ISL with Python, Chapter 2](#)

Going beyond the simple linear model

Going beyond the simple linear model

What we will cover more closely:

- Generalized Linear Model (GLM)
- General Additive Model (GAM)
- Gradient Boosted Decision Trees (GBDTs or often GBMs)

Note

You will cover more methods and details in the course *Machine Learning in Economics*. We focus here on these methods, as they are often used in industry settings when facing large amounts of tabular data.

Generalized Linear Models (GLMs)

Let's consider the scenario of log-transforming the target variable to get a more symmetric distribution to then use a simple linear model:

- Predictions and inference are in terms of logs and we have to be careful when transforming them back (Jensen's inequality).
- Interpretation changes as now unit changes in features are associated with changes in the mean of the log of Y .
- Sometimes we have 0s in our data and would have to use offsets before log-transforming.

Let's generalize the linear model

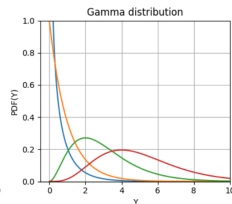
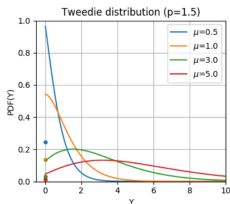
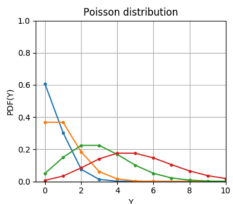
So instead of transforming the dependent variable, we can transform our linear predictor:

$$Y = g^{-1}(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p),$$

where g^{-1} is the inverse of the distribution's link function.

Distributions

Distribution	Target Domain	Unit Deviance $d(y, \hat{y})$
Normal	$y \in (-\infty, \infty)$	$(y - \hat{y})^2$
Bernoulli	$y \in \{0, 1\}$	$2(y \log \frac{y}{\hat{y}} + (1 - y) \log \frac{1-y}{1-\hat{y}})$
Categorical	$y \in \{0, 1, \dots, k\}$	$2 \sum_{i \in \{0, 1, \dots, k\}} I(y = i) y_i \log \frac{I(y=i)}{I(\hat{y}=i)}$
Poisson	$y \in [0, \infty)$	$2(y \log \frac{y}{\hat{y}} - y + \hat{y})$
Gamma	$y \in (0, \infty)$	$2(\log \frac{\hat{y}}{y} + \frac{y}{\hat{y}} - 1)$
Inverse Gaussian	$y \in (0, \infty)$	$\frac{(y - \hat{y})^2}{y \hat{y}^3}$



Source: [An Introduction to Statistical Learning](#)

Link functions

The inverse link function is the function used to transform the linear predictor to the domain of the target variable, e.g. $0 < y < 1$, or $0 < y$. In principle there are various choices of inverse link functions.

Each distribution has a so-called canonical link function, which **represents the link between the linear predictor and the expectation of the response variable**.

When forming the log-likelihood, the canonical link becomes apparent. Poisson example with log-link:

$$\log(P(y|\lambda)) = \log\left(\frac{\lambda^y e^{-\lambda}}{y!}\right) = -\log(y!) - \lambda + y\log(\lambda).$$

In a Poisson regression we parametrize λ to get a conditional estimate of the rate:

$$E[y|X] = \exp(\theta'X)$$

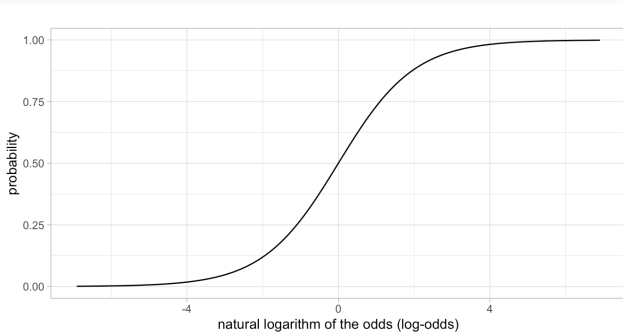
Using the **canonical link** functions gives us unbiased estimates, this is also known as **balance property**.

However, we sometimes deviate from the canonical link function for numerical reasons and interpretability reasons. For instance, the canonical link of the gamma distribution is $1/x$, but consensus is to use the log-link to get a multiplicative model and the bias is usually small and can be adjusted (e.g. via scaling).

Logistic Regression is a GLM

You have heard of Logistic regression before but probably without referring to it as a GLM. Yet, it is a GLM based on a Bernoulli distribution with a logistic (sigmoid) link function:

$$y_i = \frac{\exp(\beta_0 + \dots \beta_p x_{i,p})}{1 + \exp(\beta_0 + \dots \beta_p x_{i,p})}$$



Source: [Analysing Data using Linear Models](#)

Generalized Additive Models (GAMs)

Generalized Additive Models can be seen as an extension of the standard linear model or GLMs, where we also allow for non-linear functions for each of the variables, while keeping additivity in the linear predictor.

So instead of the simple linear model:

$$y_i = \beta_0 + \beta_1 x_{i,1} + \dots + \beta_p x_{i,p},$$

we would write:

$$y_i = \beta_0 + f_1(x_{i,1}) + \dots + f_p(x_{i,p}),$$

where f_j are separate functions (e.g. splines) for each of the features.

Generalized Additive Models (GAMs)

Advantages:

- Using non-linear functions for specific features adds a lot of flexibility and we don't have to manually try many different transformations of the feature.
- Can improve prediction accuracy by capturing non-linear effects.
- Still good degree of interpretability due to additivity.

Disadvantages:

- We have to be more careful about overfitting (yet, we should always be careful)
- We still have to pre-define potential interactions between features.

Gradient Boosted Decision Trees (GBDTs or GBMs)

Gradient Boosted Decision Trees (GBDTs or GBMs) prove very effective in real-world applications on tabular data, which is also supported by studies comparing various algorithms:

- Tabular Data: Deep Learning is not all you need.
- When do NNs outperform Boosted Trees?

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \dots, B$, repeat:
 - (a) Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data (X, r) .
 - (b) Update $\hat{f}$ by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \quad (8.10)$$

- (c) Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

3. Output the boosted model,

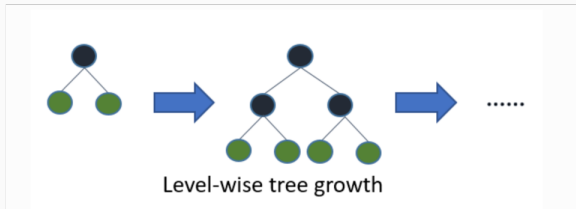
$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

Source: [An Introduction to Statistical Learning](#)

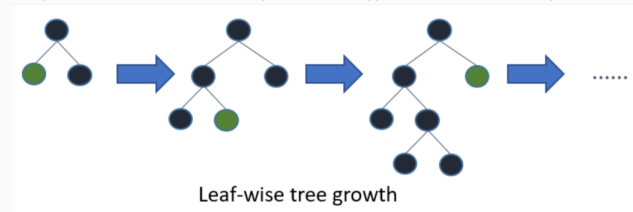
Popular GBM libraries

The most popular gradient boosting libraries, often fighting for the win in Kaggle competitions are **XGBoost** and **LGBM**. They offer many of the same features but differ in their growth strategy:

Most GBM algorithms such as XGBoost use depth-wise growth



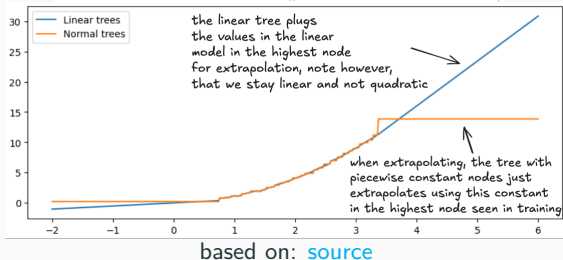
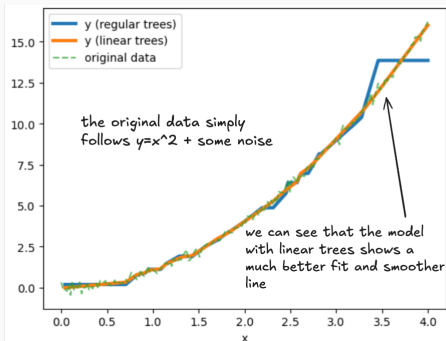
LGBM uses leaf-wise growth (best-first).
Holding number of leafs fixed, this growth strategy shows better accuracy



One criticism about tree-based algorithm is often that they don't provide a good fit for smooth functions, or that we would need many trees to approximate a simple linear function given their piece-wise constant leaf estimates.

This can be solved by using piecewise-linear trees , see [Paper](#). Instead of a constant per node, we estimate a linear model (based on numeric features).

Extensions: Linear Trees



Scikit-learn's estimator API

Scikit-learn's estimator API

Given scikit-learn's omnipresence in data science related work, it makes sense to take a closer look into some of the API decisions to better understand the overarching framework, which also impacts many downstream projects. See: [scikit-learn API](#). It follows the following concepts:

- **Consistency:** All objects share a consistent interface composed of a limited set of methods.
- **Inspection:** Learning algorithm parameters are stored as public attributes for accessibility
- **Limited object hierarchy:** Learning algorithms are the only objects to be represented using custom classes.
- **Composition:** Option to combine multiple models.
- **Sensible defaults:** When models require user-specified parameters, the library defines an appropriate default value.

Working with model pipelines

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
```

```
# Load the California housing dataset
data = fetch_california_housing(as_frame=True)
X, y = data.data, data.target
```

```
# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Define the preprocessing steps using a ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ('scaler', StandardScaler(), X.columns),
    ('poly', PolynomialFeatures(degree=2), ["AveBedrms", "Population"])
])
```

Some simple FE steps
combined in a
ColumnTransformer

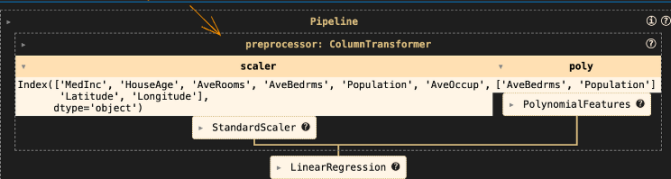
```
# Create the pipeline with preprocessing and Linear Regression estimator
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])
```

Wrapping pre-processing and
Estimator in Pipeline

pipeline

✓ 0.0s

We can visualise the Pipeline to see whether
the steps are as defined



Working with model pipelines

```
# Fit the pipeline on the training data
pipeline.fit(X_train, y_train)
```

to estimate the model
we simply call the `fit` method
and provide the training data

```
# Predict using the pipeline on the test data
y_pred = pipeline.predict(X_test)
```

✓ 0.0s

notice that we didn't have to assign
the pipeline, as the fitted parameters
are stored within the class objects

```
pipeline[-1].coef_
```

✓ 0.0s

```
array([ 8.58239564e-01,  1.25375342e-01, -2.94269358e-01,  2.26441864e-01,
        -3.39726344e-07, -3.95868459e-02, -9.05163959e-01, -8.80836212e-01,
         2.77555756e-17,  9.80949383e-02, -3.86276310e-04,  6.16799497e-03,
         3.54016233e-04,  6.46140058e-10])
```

```
pipeline[-1].intercept_
```

✓ 0.0s

```
np.float64(1.9632617385844202)
```

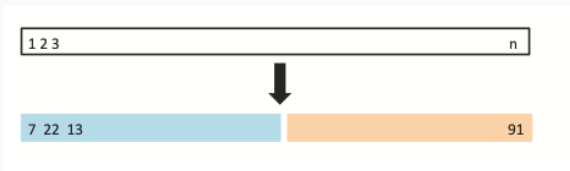
we can access the parameters
from the estimator as the
last step in the pipeline, fitted
params are followed by a
`_` suffix

Cross-Validation

Cross-Validation

It is important to distinguish between the training error rate and the test or validated error rate. The former can massively underestimate the latter if we overfit.

The simplest approach is to construct a test or validation set:

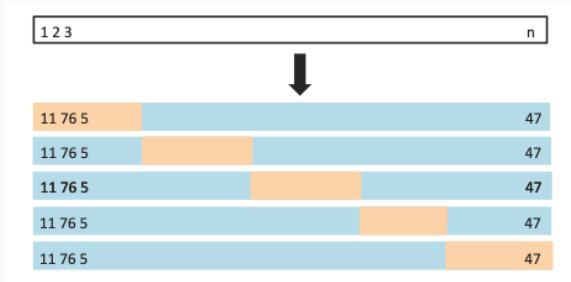


However, while simple, this has two drawbacks:

- The test error depends on the composition of the test set and we could potentially also overfit on the specific test set.
- We don't use all the available data for training.

Cross-Validation

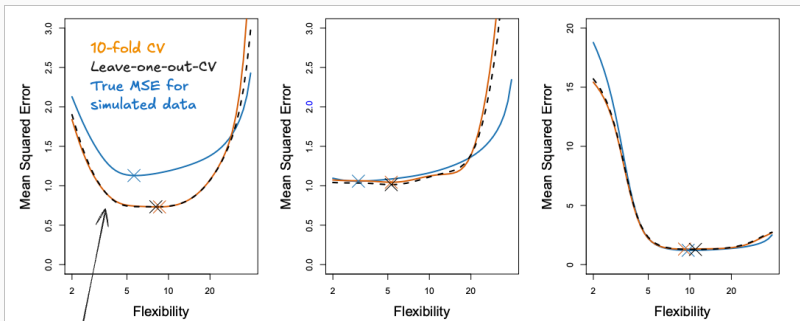
To make the most of our training data and also use multiple test sets to get a robust test error rate, we can use **k-fold** cross-validation.



We always keep one fold as validation set and train on the remaining $k - 1$ folds. The remaining cross-validated error, e.g. MSE, is then simply the average over all k fits:

$$CV_{(k)} = 1/k \sum_i^k MSE_i$$

Cross-Validation



while we still under-estimate the true test MSE here, we consistently find the best models at the minimum of the CV MSE in all three simulation scenarios

Source: [ISL with Python, Chapter 5](#)

Cross-Validation using scikit-learn

```
# Initialize default LGBMRegressor
lgb = LGBMRegressor(objective="gamma")
```

```
# Define hyperparameters for tuning
param_grid = {
```

```
    'regressor__n_estimators': [100, 200, 300],
    'regressor__learning_rate': [0.01, 0.02, 0.05, 0.1, 0.2]
}
```

parameter grid we want to search over

```
# Use GridSearchCV for hyperparameter tuning
grid_search = GridSearchCV(lgb, param_grid, cv=5)
grid_search.fit(X_train, y_train)
```

define grid search using 5 folds

```
# Print best params
print(grid_search.best_params_)
```

We can access the best parameters and the best estimator as part of the grid search object

```
# Get the best estimator
best_model = grid_search.best_estimator_
```

```
# Predict using the best model on the test data
y_pred = best_model.predict(X_test)
```

here we hit a border of our search space, so we should probably adjust the grid

✓ 46.4s

```
{'regressor__learning_rate': 0.01, 'regressor__n_estimators': 100}
```

Case study: French Motor Third-Party Liability Claims

Case study: French Motor Third-Party Liability Claims

The following case study is based on one of [glum's](#) (QuantCo's open source GLM library) [tutorials](#).

Background: Insurance claims are requests made by a policy holder to an insurance company for compensation in the event of a covered loss. When modelling these claims, the goal is often to estimate, per policy, the total claim amount per exposure unit. (i.e. number of claims average amount per claim per year). This amount is also referred to as the pure premium.

Two approaches for modelling this value are:

- Modelling the total claim amount per exposure directly
- Modelling number of claims and claim amount separately with a frequency and a severity model

Case Study: Claims Data

```
df = load_transform()
with pd.option_context('display.max_rows', 10):
    display(df)
```

✓ 4.1s

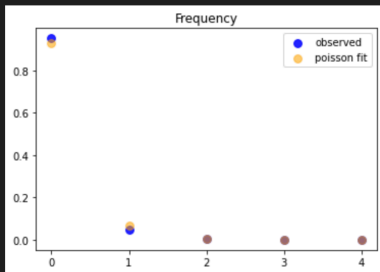
	ClaimNb	Exposure	Area	VehPower	VehAge	DrivAge	BonusMalus	VehBrand	VehGas	Density	Region	ClaimAmount	ClaimAmountCut
IDpol													
1	0	0.10000	D	5	0	5	50	B12	Regular	1217	R82	0.0	0.0
3	0	0.77000	D	5	0	5	50	B12	Regular	1217	R82	0.0	0.0
5	0	0.75000	B	6	1	5	50	B12	Diesel	54	R22	0.0	0.0
10	0	0.09000	B	7	0	4	50	B12	Diesel	76	R72	0.0	0.0
11	0	0.84000	B	7	0	4	50	B12	Diesel	76	R72	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
6114326	0	0.00274	E	4	0	5	50	B12	Regular	3317	R93	0.0	0.0
6114327	0	0.00274	E	4	0	4	95	B12	Regular	9850	R11	0.0	0.0
6114328	0	0.00274	D	6	1	4	50	B12	Diesel	1323	R82	0.0	0.0
6114329	0	0.00274	B	4	0	5	50	B12	Regular	95	R26	0.0	0.0
6114330	0	0.00274	B	7	1	2	54	B12	Diesel	65	R72	0.0	0.0

678013 rows x 13 columns

A frequency model for the claims count I

```
# plt.subplots(figsize=(10, 7))
df_plot = (
    df.loc[:, ['ClaimNb', 'Exposure']].groupby('ClaimNb').sum()
    .assign(Frequency_Observed = lambda x: x.Exposure / df['Exposure'].sum())
)
mean = df['ClaimNb'].sum() / df['Exposure'].sum()

x = range(5)
plt.scatter(x, df_plot['Frequency_Observed'].values, color="blue", alpha=0.85, s=60, label='observed')
plt.scatter(x, scipy.stats.poisson.pmf(x, mean), color="orange", alpha=0.55, s=60, label="poisson fit")
plt.xticks(x)
plt.legend()
plt.title("Frequency");
```



A frequency model for the claims count II

```
z = df['ClaimNb'].values
weight = df['Exposure'].values
y = z / weight # claims frequency

ss = ShuffleSplit(n_splits=1, test_size=0.1, random_state=42)
train, test = next(ss.split(y))

categoricals = ["VehBrand", "VehGas", "Region", "Area", "DrivAge", "VehAge", "VehPower"]
predictors = categoricals + ["BonusMalus", "Density"]
glm_categorizer = Categorizer(columns=categoricals)

X_train_p = glm_categorizer.fit_transform(df[predictors].iloc[train])
X_test_p = glm_categorizer.transform(df[predictors].iloc[test])
y_train_p, y_test_p = y[train], y[test]
w_train_p, w_test_p = weight[train], weight[test]
z_train_p, z_test_p = z[train], z[test]
```

A frequency model for the claims count III

```
f_glm1 = GeneralizedLinearRegressor(family='poisson', alpha_search=True, l1_ratio=1, fit_intercept=True)
```

```
f_glm1.fit(  
    X_train_p,  
    y_train_p,  
    sample_weight=w_train_p  
);
```

```
pd.DataFrame({'coefficient': np.concatenate(([f_glm1.intercept_], f_glm1.coef_)),  
             index=['intercept'] + f_glm1.feature_names_.T
```

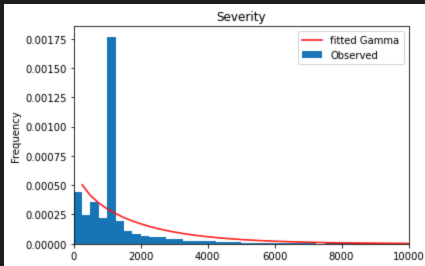
✓ 19.3s

	intercept	VehBrand__B1	VehBrand__B10	VehBrand__B11	VehBrand__B12	VehBrand__B13	VehBrand__B14	VehBrand__B2
coefficient	-4.269268	-0.003721	-0.010846	0.138466	-0.259298	0.0	-0.110712	-0.003604

1 rows x 9 columns

A severity model for the claims amount I

```
df_plot = (  
    df.loc[:, ['ClaimAmountCut', 'ClaimNb']]  
    .query('ClaimNb > 0')  
    .assign(Severity_Observed = lambda x: x['ClaimAmountCut'] / df['ClaimNb'])  
)  
  
df_plot['Severity_Observed'].plot.hist(bins=400, density=True, label='Observed', )  
  
x = np.linspace(0, 1e5, num=400)  
plt.plot(x,  
         scipy.stats.gamma.pdf(x, *scipy.stats.gamma.fit(df_plot['Severity_Observed'], floc=0)),  
         'r-', label='fitted Gamma')  
plt.legend()  
plt.title("Severity");  
plt.xlim(left=0, right = 1e4);  
#plt.xticks(x);
```



A severity model for the claims amount II

```
idx = df['ClaimAmountCut'].values > 0

z = df['ClaimAmountCut'].values
weight = df['ClaimNb'].values
# y = claims severity
y = np.zeros_like(z) # zeros will never be used
y[idx] = z[idx] / weight[idx]

# we also need to represent train and test as boolean indices
itrain = np.zeros(y.shape, dtype='bool')
itest = np.zeros(y.shape, dtype='bool')
itrain[train] = True
itest[test] = True
# simplify life
itrain = idx & itrain
itest = idx & itest

X_train_g = glm_categorizer.fit_transform(df[predictors].iloc[itrain])
X_test_g = glm_categorizer.transform(df[predictors].iloc[itest])
y_train_g, y_test_g = y[itrain], y[itest]
w_train_g, w_test_g = weight[itrain], weight[itest]
z_train_g, z_test_g = z[itrain], z[itest]
```

A severity model for the claims amount III

```
s_glm1 = GeneralizedLinearRegressor(family='gamma', alpha_search=True, l1_ratio=1, fit_intercept=True)
s_glm1.fit(X_train_g, y_train_g, sample_weight=weight[itrain])

pd.DataFrame({'coefficient': np.concatenate(([s_glm1.intercept_], s_glm1.coef_)),
              index=['intercept'] + s_glm1.feature_names_).T
```

	intercept	VehBrand__B1	VehBrand__B10	VehBrand__B11	VehBrand__B12	VehBrand__B13	VehBrand__B14	VehBrand__B2
coefficient	7.3389	-0.034591	0.040528	0.13116	0.035838	0.100753	-0.073995	-0.033196

1 rows x 60 columns

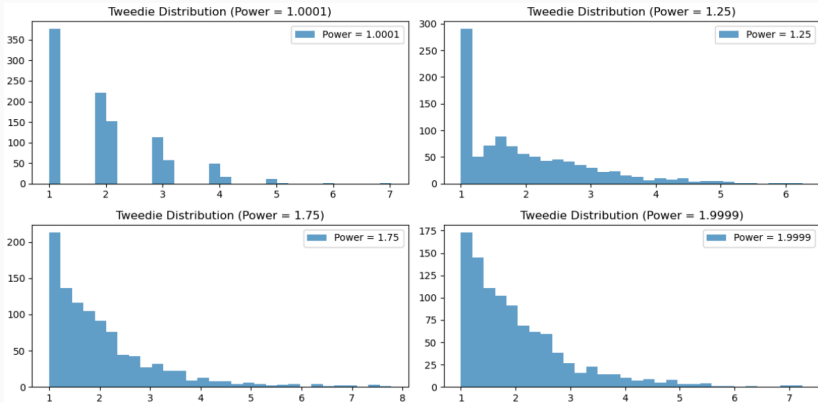
Combined GLM - Tweedie distribution I

Tweedie distributions are a special case of Exponential Dispersion Models (EDM) and nest continuous distributions such as the normal, gamma and inverse Gaussians, the discrete Poisson distribution and compound Poisson-gamma distributions.

Depending on the Tweedie power we get different distributions:

Power	Distribution
0	Normal
1	Poisson
(1,2)	Compound Poisson Gamma
2	Gamma
3	Inverse Gaussian

Tweedie illustrated



Combined GLM - Tweedie distribution II

```
weight = df['Exposure'].values
df["PurePremium"] = df["ClaimAmountCut"] / df["Exposure"]
y = df["PurePremium"]

X_train_t = glm_categorizer.fit_transform(df[predictors].iloc[train])
X_test_t = glm_categorizer.transform(df[predictors].iloc[test])
y_train_t, y_test_t = y.iloc[train], y.iloc[test]
w_train_t, w_test_t = weight[train], weight[test]
```

Combined GLM - Tweedie distribution III

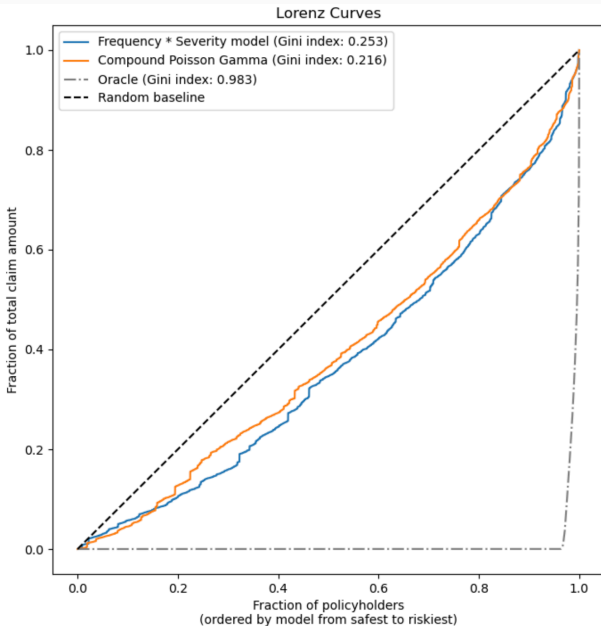
```
TweedieDist = TweedieDistribution(1.5)
t_glm1 = GeneralizedLinearRegressor(family=TweedieDist, alpha_search=True, l1_ratio=1, fit_intercept=True)
t_glm1.fit(X_train_t, y_train_t, sample_weight=w_train_t)

pd.DataFrame({'coefficient': np.concatenate(([t_glm1.intercept_], t_glm1.coef_)),
              index=['intercept'] + t_glm1.feature_names_).T
```

	intercept	VehBrand__B1	VehBrand__B10	VehBrand__B11	VehBrand__B12	VehBrand__B13	VehBrand__B14
coefficient	2.88667	-0.064157	0.0	0.231868	-0.211061	0.054979	-0.270346

1 rows x 60 columns

Compare approaches



Weekly dose of SWE: Testing & Debugging

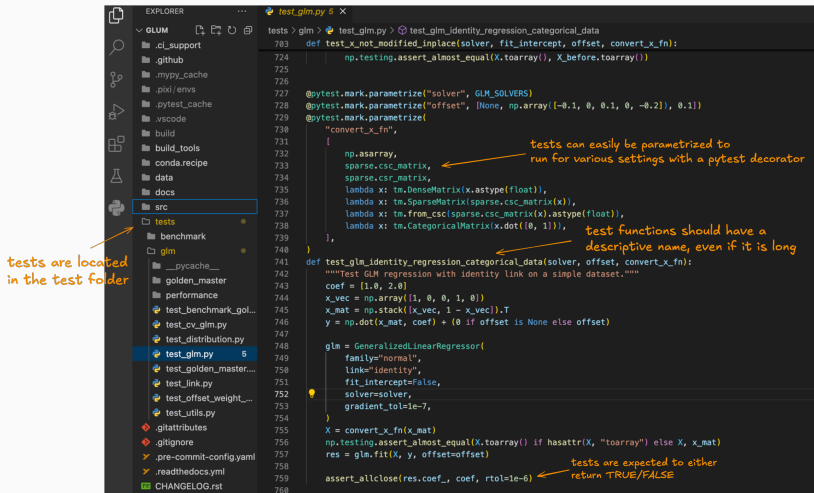
Weekly dose of SWE: Testing & Debugging

Testing is integral for creating a sustainable and robust code base in your data science project. We typically find three types of tests in a data science project:

- **Unit tests:** Test individual functions, classes, or methods in isolation to ensure they work as expected.
- **Data validation tests:** Validate data pipelines given some expectations (e.g see [Great Expectations](#)).
- **Integration tests:** Check how different modules or components work together as a whole.
- **Regression tests:** Ensure new changes do not adversely affect existing functionality.

Debugging helps you to find the reason why something doesn't work the way it should. By debugging often you become better at navigating the code base and understand the inner workings of the libraries, which eventually makes you a better coder overall.

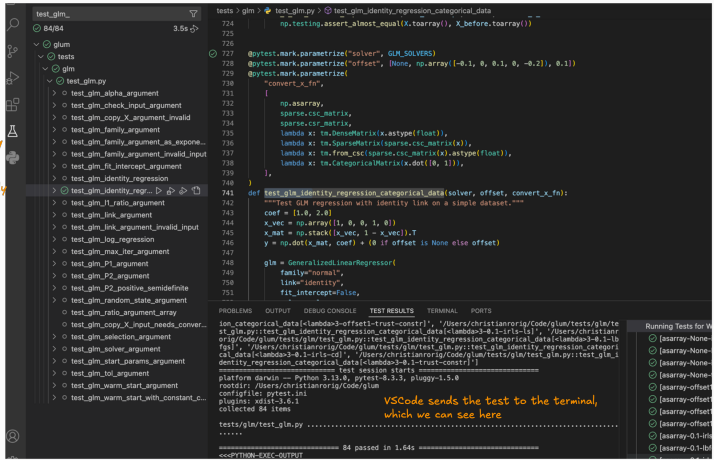
Test module



Run tests from VSCode UI

You can run tests directly from VSCode UI under the flask symbol

You can select specific tests to run or debug



The screenshot displays the VSCode interface with the following components:

- Left Sidebar:** A file explorer showing a project structure. The file `test_glm.py` is selected, and its contents are listed below it.
- Main Editor:** Displays the source code of `test_glm.py`. The code includes imports, a `pytest.mark.parametrize` decorator, and a `def test_glm_identity_regression_categorical_data` function.
- Bottom Panel:** The **TERMINAL** tab is active, showing the output of the test run. The output indicates that the test passed successfully.

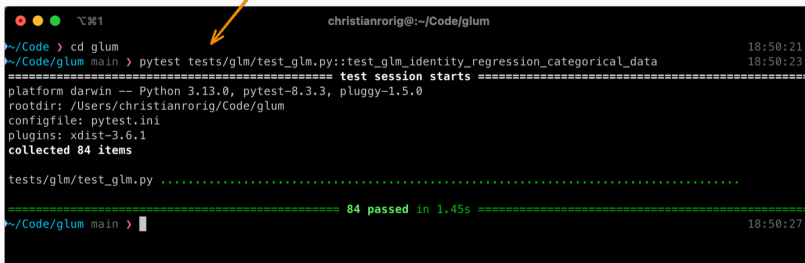
Key details from the terminal output:

- Platform: darwin
- Python: 3.13.0
- pytest: 8.3.3
- plugin: xdist-3.6.1
- collected 84 items
- tests/glm/test_glm.py (84 passed in 1.64s)

VSCode sends the test to the terminal, which we can see here

Run tests from console

We can simply run pytest with the location of the test



```
christianrorig@:~/Code/glum
~/Code > cd glm
~/Code/glum main > pytest tests/glm/test_glm.py::test_glm_identity_regression_categorical_data
===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.3.3, pluggy-1.5.0
rootdir: /Users/christianrorig/Code/glum
configfile: pytest.ini
plugins: xdist-3.6.1
collected 84 items

tests/glm/test_glm.py .....
===== 84 passed in 1.45s =====
~/Code/glum main >
```

When things don't work the way we expect them to work, we have to start debugging. Debugging can range from:

- knowing the solution directly from the error message,
- copying the error message to your favourite search engine/LLM,
- debugging yourself.

Tip: Debugging walks

Using a debugger can greatly help you to foster your understanding of the code and external libraries as it allows you to walk through the code base step by step.

Debugging in VSCode

TESTING

test_glm_

0/1 8.0s

- glum
- tests
- glm
- test_glm.py
 - test_glm_alpha_argument
 - test_glm_check_input_argument
 - test_glm_copy_X_argument_invalid
 - test_glm_family_argument
 - test_glm_family_argument_as_expone...
 - test_glm_family_argument_invalid_input
 - test_glm_fit_intercept_argument
 - test_glm_identity_regression
 - test_glm_identity_regression_categori...

test_glm.py M X

```
tests > glm > test_glm.py > test_glm_identity_regression_categorical_data
703 def test_x_not_modified_inplace(solver, fit_intercept, offset, convert_x_fn):
718
719     X_before = copy.deepcopy(X)
720     glm.fit(X, y, offset=offset)
721     if isinstance(X, np.ndarray):
722         np.testing.assert_almost_equal(X, X_before)
723     else:
724         np.testing.assert_almost_equal(X.toarray(), X_before.toarray())
725
726
727 @pytest.mark.parametrize("solver", GLM_SOLVERS)
/Users/christianrorig/Code/glum/t
/Users/christianrorig/Code/glum/tests/glm/test_glm.py::test_glm_identity_regression_categorical_data[asarra
/Users/christianrorig/Code/glum/tests/glm/test_glm.py::test_glm_identity_regression_catego
y = np.dot(x_mat, coef) + (0 if offset is None else offset)
E ValueError: shapes (5,2) and (1,) not aligned: 2 (dim 1) != 1 (dim 0)
```

In this case the error message is quite helpful and we seem to have a dimension mismatch

```
728 @pytest.mark.parametrize("offset", [None, np.array([-0.1, 0, 0.1, 0, -0.2]), 0.1])
729 @pytest.mark.parametrize(
730     "convert_x_fn",
731     [
732         np.asarray,
733         sparse.csc_matrix,
734         sparse.csr_matrix
```

PROBLEMS OUTPUT DEBUG CONSOLE TEST RESULTS TERMINAL PORTS

Debugging in VSCode

here we see the values of all the variables in scope

here we set a breakpoint to stop and investigate the code

the debugging menu helps us navigate through the code

in this case we produced the error by removing one coef

```
test_glm.py M X
tests > glm > test_glm.py > test_glm_identity_regression_categorical_data

730 "convert_x_fn",
731 [
732     np.asarray,
733     sparse.csc_matrix,
734     sparse.csr_matrix,
735     lambda x: tm.DenseMatrix(x.astype(float)),
736     lambda x: tm.SparseMatrix(sparse.csc_matrix(x)),
737     lambda x: tm.from_csc(sparse.csc_matrix(x).astype(float)),
738     lambda x: tm.CategoricalMatrix(x.dot([0, 1])),
739 ],
740 )
741 def test_glm_identity_regression_categorical_data(solver, offset, convert_x_fn):
742     """Test GLM regression with identity link on a simple dataset."""
743     coef = [1.0]
744     x_vec = np.array([1, 0, 0, 1, 0])
745     x_mat = np.stack([x_vec, 1 - x_vec]).T
746     y = np.dot(x_mat, coef) + (0 if offset is None else offset)
747
748     glm = GeneralizedLinearRegressor(
749         family="normal",
750         link="identity",
751         fit_intercept=False,
752         solver=solver,
753         gradient_tol=1e-7,
754     )
755     X = convert_x_fn(x_mat)
756     np.testing.assert_almost_equal(X.toarray() if hasattr(X, "toarray") else X, x_mat)
757     res = glm.fit(X, y, offset=offset)
```

VARIABLES

- Locals
 - > function variables
 - > coef = [1.0]
 - > offset = None
 - > solver = 'irls-ls'
 - > x_mat = array([[1, 0],
 - > x_vec = array([1, 0, 0, 1, 0])
- Globals

WATCH

- > x_mat = array([[1, 0],
- > coef = [1.0]

CALL STACK

Paused on breakpoint

- test_glm_identity_regression_categorical_data

Statistical Learning: [An Introduction to Statistical Learning, with Applications in Python](#)

GLMs:

- [medium article: Primer on GLMs](#)
- [Glum library with various tutorials](#)

Testing: [pytest documentation](#)